SPRINT 7 · MODELOS DE REGRESIÓN Y CLASIFICACIÓN · S07.1

Regresión lineal, polinómica y regularización

El gerente de TurisData quiere una tarifa de referencia: cuánto debería costar una noche según el tipo de habitación y la época. Predecir un **número** es regresión.

La idea: una suma ponderada

`precio ≈ base + (efecto suite)·es_suite + (efecto temporada alta)·temporada_alta + …`

- Cada **coeficiente** dice cuántos **euros** cambia el precio al subir esa variable en una unidad.
- Las categóricas se pasan a 0/1 con `drop_first=True`: se quita una categoría (`estandar`) que hace de **referencia**.
- Se lee: «una suite cuesta X € más que una estándar».

```
modelo_lin = LinearRegression().fit(X_train, y_train)        # Ejercicio 1
coefs = pd.Series(modelo_lin.coef_, index=X.columns)         # Ejercicio 2
```

Interpretar y revisar: coeficientes y residuos

- `extra_suite` y `efecto_temporada_alta` (≈ 22 €) salen directamente de `coefs`.
- **Residuo** = real − predicho. Un modelo bien planteado deja residuos **sin patrón** alrededor de 0.
- Curvas o un **embudo** en los residuos indican que falta algo: interacciones, transformar el objetivo (logaritmo)…

```
con_interacciones.fit(X_train, y_train)                    # Ejercicio 3
mejora_mae = mae_test - mean_absolute_error(y_test, pred_inter)
```

El coeficiente de `temporada_alta` no es el mismo «extra» para una estándar que para una suite: eso es una **interacción**.

Polinomios: captar curvas… y su peligro

El precio sube en invierno y verano y baja en primavera y otoño: relación con curvas respecto a `mes_llegada`.

```
make_pipeline(PolynomialFeatures(2), LinearRegression())          # grado 2
cross_val_score(modelo, X, y, cv=5, scoring="r2").mean()        # Ejercicio 4
```

Opción	Idea
Polinomio de grado alto	imita la curva, pero es frágil y difícil de explicar
`temporada_alta` (variable pensada)	simple, explicable y suele ganar

> Pensar bien las variables (*feature engineering*) casi siempre supera a añadir complejidad al modelo.

Muchas variables, pocos datos (cuaderno S07_02)

Variables originales **más todas sus combinaciones de grado 2**: cientos de columnas para solo 120 filas. Solo unas pocas influyen de verdad.

```
r2_train_ols = ols.score(X_train, y_train)       # Ejercicio 1
r2_test_ols  = ols.score(X_test, y_test)
brecha_ols = r2_train_ols - r2_test_ols          # ¿es una brecha sana?
```

Una brecha grande entre entrenamiento y prueba es **sobreajuste**. La solución: penalizar los coeficientes grandes.

Ridge (L2) y Lasso (L1)

Ridge

Penaliza `alpha · Σ coef²`. Encoge todos los coeficientes; reparte el peso entre variables parecidas. Rara vez deja alguno en cero.

Lasso

Penaliza `alpha · Σ |coef|`. **Lleva a cero** las variables poco útiles: modelo disperso y selección de variables.

ElasticNet

Mezcla ambas: útil con variables muy correlacionadas.

```
ridge = modelo_con(RidgeCV(alphas=np.logspace(-1, 4, 30))).fit(X_train, y_train)   # Ejercicio 2
lasso[-1].coef_                                                                      # Ejercicio 3
```

Elegir `alpha` y cuántos datos hacen falta

- `RidgeCV` y `LassoCV` prueban una lista de `alpha` con validación cruzada **dentro de train**; el test no se toca hasta el final.
- **Escalar antes** de penalizar: si no, las variables con números grandes quedan castigadas injustamente.
- Lasso gana cuando **pocas** variables lo explican casi todo (nuestro caso) y es fácil de explicar al cliente («las 5 cosas que más influyen en el precio»).
- Mini-reto: `curva` con el R^2 en test de `ols` y `lasso` para `tamanos = [60, 120, 300, 1000]`. Con muchos datos, regularizar importa menos.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S07_01_regresion_lineal_polinomica — 40 min

Regresión lineal y polinómica

sprints/S07/notebooks/S07_01_regresion_lineal_polinomica.ipynb

CUADERNO

S07_02_ridge_lasso — 35 min

Ridge y Lasso

sprints/S07/notebooks/S07_02_ridge_lasso.ipynb